Who Writes the Testbench Matters: Auditing False Rejections in LLM-Generated Verilog Testbenches

Soham Guha
School of Computer Engineering
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, Odisha, India
Email: soham.guha@kiit.ac.in

***Abstract*—Closed-loop LLM flows for register-transfer level (RTL) design increasingly let a language model write the testbench that judges a candidate circuit. If that testbench rejects a correct design, the loop is steered away from correct hardware. We audit 1,022 free-form, self-checking Verilog testbenches written by GPT-4o-mini and Gemini Flash-Lite (plus a 20-trial local-model pilot) for 30 VerilogEval-v2 spec-to-RTL problems, and measure how often a testbench that compiles and obeys its output contract nevertheless fails the golden reference design. This *false rejection* occurs in 58.32% of all trials (596/1,022; problem-level cluster bootstrap 95% CI [48.69%, 67.72%]) and in 60.51% of testbenches that pass the compile-and-contract gate (596/985). All 1,022 verdicts reproduce exactly when re-simulated under Icarus Verilog 12.0, and an independent re-simulation under Verilator 5 confirms 566 of the 592 false rejections it could build (95.6%). Six prompting conditions—self-testing, an adversarial "red-team" prompt with and without the candidate RTL, the same prompts given to the other model, and an independent resample of the self-test—all fall between 55.7% and 62.9% ($\chi^2 = 3.15$, $p = 0.68$). In contrast, the identity of the model that writes the testbench is decisive: GPT-4o-mini testbenches falsely reject 74.3% of the time versus 42.6% for Gemini Flash-Lite (paired per-problem difference -31.2 percentage points, 95% CI [-42.6, -20.2]; problem-clustered GEE odds ratio 4.13, [2.50, 6.81]). Once the tester is fixed, the identity of the model that wrote the candidate RTL has no measurable effect, and an adversarial prompt does not reduce—and may slightly increase—false rejections. False rejections are concentrated in sequential circuits (68.0% vs. 33.8% for combinational) and vary from 0% to 94% across problems (ICC $= 0.27$). In 31 of 194 cases where the tester saw a *buggy* candidate, the resulting testbench rejected the golden design while accepting the buggy one. We release the audit scripts and per-trial re-simulation results.**

***Index Terms*—Hardware verification, large language models, Verilog RTL, testbench generation, false rejection, empirical study.**

I. Introduction

Large language models (LLMs) are now routinely evaluated as RTL authors [1]–[4]. Feedback-driven frameworks such as AutoChip [5] close the loop by simulating LLM-generated Verilog against a testbench and returning compiler and simulation errors to the model. AutoChip assumes the testbench is supplied and correct; more recent agentic systems remove this assumption by having LLM agents generate the testbench as well [6]–[8].

This shift makes the reliability of the testbench a first-order concern. In hardware, escaped functional bugs are expensive [9], but the opposite error is also costly inside an automated loop: a testbench that *falsely rejects* a correct design sends the generator into spurious repair iterations and, if its expectations encode a misreading of the specification, actively pushes the design toward that misreading. Whether a second model, a different model family, or an adversarial prompt reduces such errors is an empirical question; it is motivated by evidence that errors of different LLMs are correlated, especially within a provider [10], and by multi-agent debate results in reasoning tasks [11].

We audit 1,022 free-form, self-checking Verilog testbenches for 30 VerilogEval-v2 problems [1], [2]. Each testbench passes through three gates: (1) it must compile with the golden RTL under Icarus Verilog 12.0 (`-g2012`) and print exactly one standardized PASS/FAIL line before `$finish`; (2) it must report PASS on the golden reference RTL; and (3) it must report FAIL on an inverted-output version of the golden RTL (non-vacuity). A trial is a *false rejection* when

$$F_i = C_i \wedge K_i \wedge \neg G_i, \tag{1}$$

where C_i denotes successful compilation, K_i contract compliance on the golden-RTL simulation, and G_i a PASS verdict on the golden RTL.

We ask four questions:

- **RQ1 (Prevalence):** How often does an LLM-written testbench falsely reject the golden design, and is the measurement robust to the simulator?
- **RQ2 (Prompting role):** Do adversarial prompts, hiding the candidate RTL, or using the other model as checker reduce false rejections relative to self-testing?
- **RQ3 (Problem structure):** How do false rejections vary with circuit type and with specification defects?
- **RQ4 (Tester identity and anchoring):** How much does the identity of the testbench-writing model matter, and does seeing a buggy candidate anchor the testbench to the bug?

Our main findings are: (i) false rejection is the most common testbench outcome (58%), and the verdicts are simulator-robust; (ii) prompting role has no detectable effect, and an exact resample of the self-test condition differs by only -1.4 percentage points, giving a direct estimate of sampling noise; (iii) the testbench-writing model dominates every other factor we measured; and (iv) testbenches written while viewing

buggy RTL sometimes certify the bug while rejecting the correct design, which is the most damaging failure for a repair loop.

II. Related Work

A. LLMs for RTL Generation and Verification

VerilogEval [1] and its spec-to-RTL revision [2], as well as RTLLM [3], evaluate LLM-generated Verilog by simulating it with fixed, benchmark-supplied testbenches against reference solutions. Chip-Chat [4] explored conversational design with a human in the loop, and AutoChip [5] automated the loop by feeding compiler and simulation results from provided testbenches back to the LLM. In all of these the oracle is human-authored.

Several works generate the verification artifacts themselves. AssertLLM [12] generates SystemVerilog assertions from specification documents. AutoBench [7] generates hybrid Python/Verilog testbenches and evaluates them against golden RTL and RTL mutants; CorrectBench [8] adds LLM-based self-validation and self-correction of the testbench and reports that sequential tasks remain substantially harder. MAGE [6] uses separate testbench-generation, RTL-generation, judge, and debug agents, so the judge must decide whether the RTL or the testbench is at fault when they disagree. Our audit is complementary: rather than proposing a generator, we measure how often a generated testbench rejects a *known-correct* design and which experimental factors move that rate.

B. Correlated Errors and Multi-Model Checking

Multi-agent debate improves factuality and reasoning on several benchmarks [11]. Kim et al. [10] show that LLM errors are substantially correlated across models, more so for models sharing a provider or architecture, which motivates checking one model's work with a model from another family. We test this idea directly for Verilog testbenches by comparing self-testing, same-model adversarial prompts, and cross-model checking under matched prompts.

III. Experimental Methodology

A. Benchmark Problems

We screened the first 122 of the 156 VerilogEval-v2 spec-to-RTL problems [2] and retained those whose golden RTL yields at least eight compilable, non-equivalent mutants (equivalence judged by a 2,000-vector differential simulation); 35 problems qualified and the first 30 in benchmark order were used. This filter removes trivial single-gate problems but also biases the suite toward sequential logic: by the presence of clock edges in the reference RTL, 8 problems are combinational (vector/mux/arithmetic logic and three FSM next-state-logic problems) and 22 are sequential (counters, LFSRs, shift registers, and FSMs). Each problem provides a natural-language specification, a golden reference RTL, and the official VerilogEval testbench. We confirmed that every golden RTL passes its official testbench with zero mismatches under Icarus Verilog 12.0.

B. Execution Environment

All archived simulations used Icarus Verilog 12.0 (`iverilog -g2012`, `vvp`) with a 30-second compile and simulation timeout. For the audit we re-simulated every free-form testbench with Icarus Verilog 12.0 on Linux and, as an independent second engine, with Verilator 5.020 in `--binary --timing` mode, which supports delay and event controls in testbench processes [13].

C. Generation Protocol

Testbenches were requested under a fixed contract: module `tb`, DUT instance `TopModule dut`, outputs compared with `!==`, a standardized `MISMATCH` line per failed check, and exactly one `*** TEST RESULT: PASS ***` or `FAIL` line before `$finish`. If the first attempt failed to compile or violated the contract on the golden RTL, the model received the error and one retry (used in 75 of 1,022 trials). Candidate RTL was generated once per (problem, generator, sample) with one compile-error retry and reused across conditions. All models used temperature 0.2 and a 2,048-token output limit. Each configuration was sampled three times; the sample labels (42, 43, 44) enter the response-cache key but are not passed to the OpenAI or Gemini APIs, so the three samples are independent draws rather than seeded replays.

D. Archived Arms

The archive contains 1,112 trials in two arms. In a *template pilot* ($N = 90$) the harness supplied the module shell, clock, reset, and DUT instance, and the model wrote only stimulus statements; 75 testbenches failed to compile and the remaining 15 passed the golden RTL but were vacuous, so the arm produced no valid testbench and no false rejection and is excluded from analysis. The *free-form arm* ($N = 1{,}022$), in which the model writes the complete testbench, is the evaluation corpus.

E. Conditions

Let G be the model that generated the candidate RTL and T the other cloud model. The six conditions are:

- **A (Self-test):** G writes the testbench from the specification and candidate RTL using a neutral "verification engineer" prompt.
- **B (Same-model adversary):** G itself, prompted as an adversarial red-team engineer, sees specification and candidate RTL. (The configuration file names this "same-family", but for both cloud pairings G' is the same model checkpoint as G; B therefore differs from A only in the prompt.)
- **B′ (Same-model adversary, spec only):** as B, with the candidate RTL withheld.
- **C (Cross-model adversary):** T with the adversarial prompt, specification, and candidate RTL.
- **C′ (Cross-model adversary, spec only):** as C, with the candidate RTL withheld.
- **D (Self-test resample):** identical to A with an independent sample. The pipeline was designed to make D

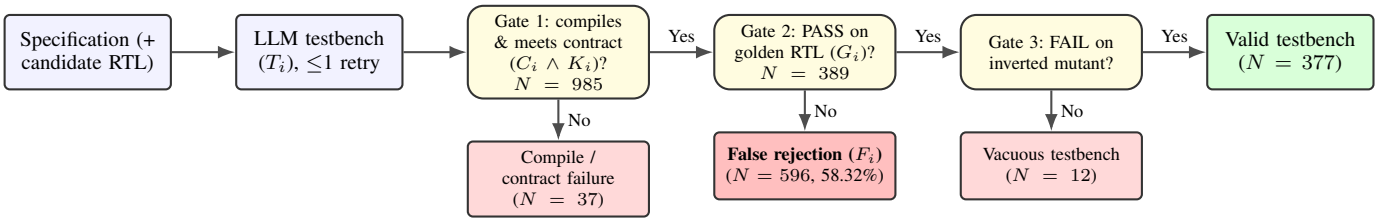


Fig. 1. Three-gate validation pipeline and outcome counts for the 1,022 free-form trials. A testbench is a false rejection (F_i) if it passes Gate 1 but reports FAIL on the golden reference RTL at Gate 2. $37 + 596 + 12 + 377 = 1{,}022$.

compute-matched to C, but as implemented it issues a single request with the same prompt and retry budget as A (mean tester tokens 1,811 vs. 1,812). We therefore treat D as a test–retest replicate of A, which directly measures sampling noise.

These conditions span three prompt types (neutral self, adversarial with RTL, adversarial spec-only) crossed with whether the testbench author is the RTL generator. Because each cloud model generates RTL for its own problems and tests the other model's RTL in C and C′, both models act as testbench authors in every condition.

F. Models and Trial Allocation

- **GPT-4o-mini** (`gpt-4o-mini-2024-07-18`): generator in 540 trials (all 30 problems) and testbench author in 514.
- **Gemini Flash-Lite** (requested as `gemini-3.5-flash-lite`): generator in 462 trials (26 problems; not Prob110, Prob111, Prob114, Prob115) and testbench author in 488 (all 30 problems). The client's fallback model `gemini-3.8-flash` served 6 generation and 5 testbench calls, and 60 trials from an early pilot have no recorded model version.
- **Local pilot:** `qwen2.5-coder:1.5b` generated RTL and `llama3.1:8b` wrote all testbenches for 20 trials on Prob006 (18) and Prob021 (2). Because the tester is `llama3.1:8b` in every condition, including the nominal self-test conditions A and D, the condition labels do not apply to these trials. They are included in prevalence estimates (RQ1, RQ3) and excluded from condition and model analyses, which use the 1,002 cloud trials.

G. Statistical Methods

Trials are clustered within 30 problems. The ANOVA estimator gives an intraclass correlation of $\rho = 0.2665$ for the false-rejection indicator; with mean cluster size $\bar{m} = 34.07$ the design effect is $D_{\text{eff}} = 1+(\bar{m}-1)\rho = 9.81$, an effective sample size of about 104 trials. We therefore report Wilson score intervals [14] only as descriptive trial-level intervals and base inference on (i) a problem-level cluster bootstrap [15], [16] (10,000 resamples of whole problems, seed 42), (ii) problem-level paired comparisons with bootstrap CIs and sign-flip permutation tests (10,000 draws), and (iii) a logistic generalized estimating equation (GEE) [17] with problem clusters and an exchangeable working correlation. The omnibus Pearson χ^2 test [18] across conditions ignores clustering and is reported for completeness; because it is anti-conservative, a non-significant result is all the more informative. All numbers in this paper are produced by `paper/audit/analysis.py`.

IV. Results

A. RQ1: Prevalence and Simulator Robustness

Of 1,022 free-form testbenches, 7 failed to compile and 30 violated the output contract on the golden RTL, leaving 985 (96.38%) that passed Gate 1 (Fig. 1). Only 389 of these reported PASS on the golden RTL, and 12 of those also passed the inverted-output mutant and were rejected as vacuous, leaving 377 valid testbenches (36.89%). The remaining 596 trials are false rejections:

$$\text{FRR}_{\text{all}} = \frac{596}{1{,}022} = 58.32\%, \qquad \text{FRR}_{\text{G1}} = \frac{596}{985} = 60.51\%, \tag{2}$$

with problem-level cluster bootstrap CIs of [48.69%, 67.72%] and [51.50%, 69.11%]. Restricting to the 1,002 cloud trials gives 58.88% (590/1,002).

Reproduction. We re-ran every free-form testbench against the golden RTL and the inverted-output mutant under Icarus Verilog 12.0 on a different host and operating system. All 1,022 archived verdicts (compile failure, contract violation, FAIL, PASS, vacuous) were reproduced exactly.

Second simulator. Of the 985 Gate-1 testbenches, 972 built and met the contract under Verilator 5.020 (10 failed to build, 3 did not terminate cleanly). The two simulators returned the same verdict for 935 of these 972 (96.2%). Of the 592 Icarus false rejections that Verilator ran, 566 (95.6%) also FAIL under Verilator, 26 PASS; conversely 11 of 380 Icarus passes FAIL under Verilator. The disagreements concentrate in a few problems (`Prob105_rotate100`: 13, `Prob100_fsm3comb`: 10) and are consistent with testbenches that sample outputs in the same time step as the clock edge, whose outcome depends on event ordering. Counting only false rejections confirmed by both simulators gives a lower bound of 566/1,022 = 55.38%; the conclusions below are unchanged under this stricter definition.

B. RQ2: Prompting Role Does Not Move the Rate

Table II gives outcomes for the 1,002 cloud trials. Rates lie between 55.69% and 62.87%, and the omnibus test is far from significance ($\chi^2 = 3.15$, $df = 5$, $p = 0.68$). Problem-level paired comparisons (30 problems) isolate individual factors:

TABLE I

FALSE-REJECTION RATES ACROSS PROBLEM SUBSETS (ALL 1,022 FREE-FORM TRIALS). "GATE 1" ROWS CONDITION ON TESTBENCHES THAT COMPILE AND MEET THE CONTRACT ($C_i \wedge K_i$, INCLUDING THE 12 VACUOUS TESTBENCHES, WHICH PASS THE GOLDEN RTL). CLUSTER BOOTSTRAP: 10,000 PROBLEM-LEVEL RESAMPLES, SEED 42.

Subset	**Problems**	**Trials** (N)	F_i	**Rate (%)**	**Wilson 95% CI**	**Cluster Bootstrap 95% CI**
All trials	30	1,022	596	**58.32**	[55.27, 61.30]	[48.69, 67.72]
Gate 1 passed	30	985	596	**60.51**	[57.42, 63.51]	[51.50, 69.11]
Excl. 2 port-inconsistent problems	28	950	539	**56.74**	[53.57, 59.85]	[46.49, 66.45]
Gate 1 passed	28	914	539	**58.97**	[55.75, 62.12]	[49.38, 67.81]
Excl. all 3 flagged problems	27	914	505	**55.25**	[52.01, 58.45]	[44.91, 64.97]
Combinational	8	290	98	**33.79**	[28.59, 39.42]	[18.39, 50.00]
Gate 1 passed	8	254	98	**38.58**	[32.81, 44.70]	[22.34, 54.08]
Sequential	22	732	498	**68.03**	[64.57, 71.31]	[59.19, 76.09]
Gate 1 passed	22	731	498	**68.13**	[64.66, 71.40]	[59.26, 76.26]
6 clean combinational (post hoc)	6	218	54	**24.77**	[19.51, 30.90]	[9.89, 40.74]
Gate 1 passed	6	187	54	**28.88**	[22.86, 35.75]	[12.35, 45.26]
4 simple combinational (post hoc)	4	146	18	**12.33**	[7.94, 18.65]	[1.79, 18.89]
Gate 1 passed	4	118	18	**15.25**	[9.87, 22.83]	[2.50, 21.79]

TABLE II

OUTCOMES BY CONDITION (1,002 CLOUD TRIALS). EACH CONDITION HAS 90 GPT-4O-MINI AND 77 GEMINI FLASH-LITE TESTBENCH AUTHORS; GATE-1 COUNTS INCLUDE VACUOUS TESTBENCHES.

Cond.	N	**Pass G1**	F_i	**Rate (%)**	**Wilson 95% CI**
A (Self)	167	159	93	55.69	[48.11, 63.01]
B (Same+RTL)	167	161	105	62.87	[55.33, 69.84]
B′ (Same-Spec)	167	164	102	61.08	[53.51, 68.14]
C (Cross+RTL)	167	159	96	57.49	[49.90, 64.73]
C′ (Cross-Spec)	167	164	101	60.48	[52.91, 67.58]
D (Resample of A)	167	161	93	55.69	[48.11, 63.01]

Pearson $\chi^2 = 3.15$, $df = 5$, $p = 0.68$. Problem-level paired permutation: B−A $+7.22$ pp ($p = 0.18$); C−B -6.78 pp ($p = 0.17$); D−A -1.44 pp ($p = 0.84$).

- *Sampling noise* (D−A, identical configuration): -1.44 pp, 95% CI $[-11.67, 7.78]$.
- *Adversarial prompt* (B−A, same model and inputs): $+7.22$ pp, $[-1.11, 16.67]$, $p = 0.18$.
- *Hiding the candidate RTL* (B′−B): -3.78 pp, $[-14.11, 6.33]$, $p = 0.54$; (C′−C): $+2.44$ pp, $[-5.11, 10.00]$, $p = 0.59$.
- *Cross-model checker* (C−B, same prompt): -6.78 pp, $[-15.78, 2.56]$, $p = 0.17$.

The A–D replicate shows that, with three samples per problem, condition-level differences of up to about ± 10 pp are within sampling noise; none of the designed contrasts exceeds that band.

C. RQ3: Circuit Type and Specification Defects

1) Combinational vs. sequential: False rejections are concentrated in sequential problems (68.03%, cluster CI [59.19%, 76.09%]) relative to combinational ones (33.79%, [18.39%, 50.00%]) (Table I). Per-problem rates range from 0% (`Prob114_bugs_case`) to 94.4% (`Prob040_count10`, `Prob068_countbcd`, `Prob088_ece241_2014_q5b`). With only eight combinational problems, this contrast is between small groups of problems and should not be read as a per-circuit causal effect.

2) Specification defects: Comparing each specification with its reference RTL, we flagged three problems:

1) `Prob068_countbcd`: the specification declares `ena` as 4 bits, while the reference declares `ena[3:1]` (3 bits); the prose ("for digits [3:1]") matches the reference.
2) `Prob099_m2014_q6c`: the specification declares a 7-bit `y` and asks for next-state signals `Y2` and `Y4`, while the reference has a 6-bit `y` and ports `Y1`, `Y3`.
3) `Prob088_ece241_2014_q5b`: the specification asks for a one-hot encoding while the reference uses a binary state register. Because state encoding is not observable at the ports, this mismatch cannot by itself cause a false rejection; we report it separately.

The two problems with port-level contradictions account for 57 of their 72 trials (79.17%). Excluding them lowers the overall rate by only 1.58 pp, to 56.74% (cluster CI [46.49%, 66.45%]); excluding all three gives 55.25%. Specification defects inflate false rejections on the affected problems, but they do not explain the overall rate.

3) Post hoc combinational subsets: Removing `Prob099` (port-inconsistent) and `Prob043_vector5` leaves six combinational problems with a 24.77% rate (54/218). `Prob043`'s specification is consistent with its reference (the bit ordering is fixed by an example), so its exclusion is a post hoc choice rather than a specification defect; including it raises the rate to 33.79% (all eight combinational problems). Two of these (`Prob091`, `Prob100`) describe FSM next-state logic; the remaining four purely datapath problems (`Prob006`, `Prob021`, `Prob033`, `Prob114`) still have a 12.33% rate (18/146, cluster CI [1.79%, 18.89%]). These subsets were defined after seeing the data and are exploratory.

D. RQ4: Tester Identity Dominates

1) Testbench author: Pooling over conditions, GPT-4o-mini testbenches falsely reject the golden design in 382 of 514 trials (74.32%, Wilson [70.37%, 77.91%]) and Gemini Flash-Lite testbenches in 208 of 488 (42.62%, [38.31%, 47.05%]). The

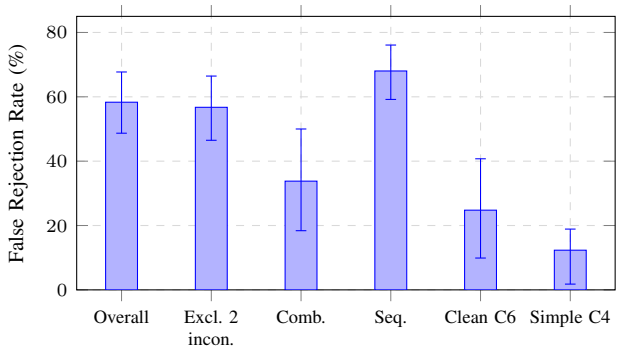


Fig. 2. False-rejection rates across problem groupings with 95% problem-level cluster bootstrap confidence intervals (all 1,022 free-form trials).

gap holds within each prompt type (Table III, Fig. 3) and at the problem level: across the 30 problems the mean per-problem rate is 72.87% vs. 41.64%, a paired difference of -31.23 pp (95% bootstrap CI $[-42.59, -20.21]$, permutation $p < 10^{-4}$); excluding the two port-inconsistent problems gives -31.28 pp ($[-42.89, -20.17]$).

A problem-clustered logistic GEE with tester identity, prompt type, and a same-model indicator (tester is also the RTL generator) gives:

- GPT-4o-mini vs. Gemini Flash-Lite tester: OR $= 4.13$ [2.50, 6.81], $p < 0.001$;
- adversarial prompt with RTL vs. neutral self prompt: OR $= 1.32$ [0.99, 1.75], $p = 0.058$;
- adversarial spec-only prompt vs. neutral self prompt: OR $= 1.35$ [1.02, 1.79], $p = 0.035$;
- tester is the RTL generator (self vs. cross-model): OR $= 1.01$ [0.78, 1.32], $p = 0.94$.

Testing one's own design rather than the other model's design therefore makes no difference once the tester's identity is accounted for, and the adversarial "red-team" framing, if anything, slightly increases false rejections.

2) RTL generator: Grouping trials by the model that generated the candidate RTL gives a smaller difference (26 shared problems: 65.81% for GPT-4o-mini vs. 55.34% for Gemini Flash-Lite, -10.47 pp, CI $[-16.67, -4.27]$). This apparent generator effect is entirely a tester effect: four of six conditions use the generator as tester. Holding the tester fixed and restricting to prompts that show the RTL, the generator of the shown RTL changes the rate by -0.64 pp for GPT-4o-mini testers (CI $[-9.62, 8.33]$) and -1.50 pp for Gemini Flash-Lite testers ($[-13.25, 9.19]$).

3) Anchoring on buggy candidates: For each testbench we also simulated the candidate RTL it was written for. Among Gate-1 testbenches written for a candidate that fails the official VerilogEval testbench, 31 of 194 written with the candidate RTL visible (16.0%) rejected the golden design *and* passed the buggy candidate, compared with 8 of 98 (8.2%) written from the specification alone. Such a testbench would lead a repair loop to accept the bug and reject the fix. A GEE with an interaction between candidate correctness and RTL visibility points in the same direction but is not significant (interaction OR $= 1.54$ [0.89, 2.68], $p = 0.12$), so we treat anchoring as a plausible but unconfirmed mechanism.

TABLE III
FALSE REJECTIONS BY TESTBENCH AUTHOR AND PROMPT TYPE (1,002 CLOUD TRIALS), k/n (%). "SELF" POOLS A AND D; GEMINI F-L = GEMINI FLASH-LITE.

Prompt	GPT-4o-mini	Gemini F-L	Diff. (pp)
Self (A, D)	125/180 (69.4)	61/154 (39.6)	-29.8
Adv. + RTL (B, C)	123/167 (73.7)	78/167 (46.7)	-26.9
Adv. spec (B′, C′)	134/167 (80.2)	69/167 (41.3)	-38.9
All	382/514 (74.3)	208/488 (42.6)	-31.7

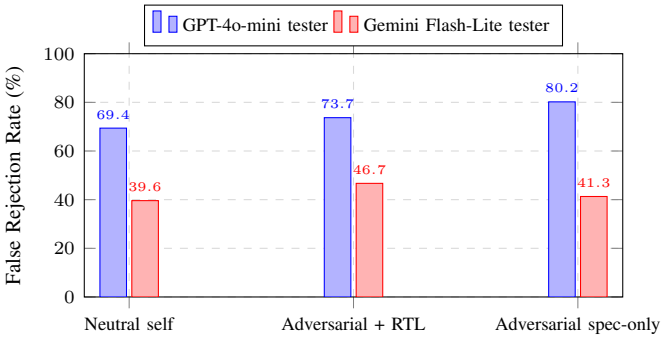


Fig. 3. False-rejection rate by testbench author and prompt type. The tester gap (about 30 pp) is several times larger than any prompt effect.

V. QUALITATIVE FAILURE ANALYSIS

Only 6 of the 596 false-rejecting testbenches emitted the requested standardized `MISMATCH` line; the rest used free-form error messages. Automated classification of the first mismatch is therefore unreliable: on a manually labeled sample of 25 false rejections, the pipeline's rule-based categories agreed with manual labels in 16 cases (64%). We therefore report only mechanisms visible directly in simulator output.

A. Cycle Alignment of Sequential Expectations

In counters and shift registers, many testbenches expect a register value one cycle early or late. For example, in `Prob040_count10` (94.4% false rejection) typical logs show the testbench's expected count sequence shifted by one or two counts relative to the golden output at every subsequent check (e.g., "Expected q = 2, got q = 1", "q = 0010, expected = 0000"), consistent with sampling outputs at the active clock edge, before the non-blocking update, or miscounting the reset cycle.

B. Checking Before Reset Takes Effect

In 52 of the 596 false rejections (8.7%), the golden design's output was still `X` at the failing check (e.g., "Expected q = 1, got q = xxxx" in `Prob035_count1to10`), meaning the testbench checked an output before applying or completing the synchronous reset. All 596 testbenches used the case-inequality operator `!==`, so these failures are sequencing errors in the stimulus, not artifacts of 2-state comparisons.

C. Under-Specified Conventions

Some specifications leave conventions implicit. `Prob060_m2014_q4k` ("a shift register with four D flops") does not state the shift direction or which flop drives `out`, and has a 61.1% false-rejection rate. In contrast, `Prob085_shift4` states the direction explicitly ("shift right (q[3] becomes zero, q[0] is shifted out)") and has one of the lowest sequential rates (19.4%), which suggests that explicit specification of conventions helps.

VI. Threats to Validity

1) *Oracle validity.* Golden RTL designs are from VerilogEval-v2 and pass their official testbenches, but they are not formally proven equivalent to the prose specifications; two problems contain port-level contradictions (Section IV-C2). In 14 false rejections the testbench also passed a candidate that passes the official testbench, which indicates either uncovered differences between candidate and golden behavior or limited coverage of the official testbench.
2) *Simulator dependence.* Archived verdicts come from Icarus Verilog. A Verilator 5 re-simulation agrees on 96.2% of the testbenches it could build; the 3.8% disagreements stem from race-sensitive sampling in testbenches, and a two-simulator-confirmed definition lowers the overall rate by about 3 pp.
3) *Model coverage and versions.* Only two cloud models were evaluated as testbench authors, so "tester identity" is a comparison of two models, not of model families. Model versions were not recorded for 60 pilot trials, and 11 calls were served by a fallback Gemini model. The 20 local-model trials are too few to support model-level conclusions.
4) *Design of conditions.* The planned compute-matched control (D) was implemented as a resample of A, so this study does not test whether a larger self-testing budget closes any gap. Condition B uses the same checkpoint as A, not a distinct same-family model.
5) *Sampling.* Three samples per configuration at temperature 0.2 provide limited within-configuration replication; the A–D replicate bounds the resulting noise at roughly ± 10 pp per condition.
6) *Problem selection.* The ≥ 8-mutant filter favors sequential problems and excludes the simplest benchmarks; prevalence estimates apply to this filtered subset.

VII. Conclusion

Across 1,022 LLM-written Verilog testbenches, 58% reject a correct reference design despite compiling and following the output contract, and this measurement is reproducible across simulators. Prompting strategies that are often proposed as remedies—adversarial red-team framing, hiding the implementation, or checking with a different model under the same prompt—do not reduce the rate beyond sampling noise. The single largest factor is which model writes the testbench: one of the two evaluated models produced false rejections at roughly 1.7 times the rate of the other. For closed-loop RTL generation, this suggests that (i) LLM-generated testbenches must themselves be validated, for example against a trusted reference model or by cross-checking several testbenches, before their verdicts are allowed to drive repairs; (ii) testbench-writing models should be benchmarked on false rejection directly, since role prompting does not substitute for a better tester; and (iii) sequential timing and reset conventions should be stated explicitly in specifications given to LLM testbench writers.

References

[1] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.

[2] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, "Revisiting VerilogEval: Newer LLMs, in-context learning, and specification-to-RTL tasks," *arXiv preprint arXiv:2408.11053*, 2024.

[3] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An open-source benchmark for design RTL generation with large language model," in *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2024.

[4] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023.

[5] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "AutoChip: Automating HDL generation using LLM feedback," *arXiv preprint arXiv:2311.04887*, 2023.

[6] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, "MAGE: A multi-agent engine for automated RTL code generation," in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2025, arXiv:2412.07822.

[7] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design," in *Proc. ACM/IEEE Int. Symp. Machine Learning for CAD (MLCAD)*, 2024.

[8] ——, "CorrectBench: Automatic testbench generation with functional self-correction using LLMs for HDL design," in *Proc. Design, Automation and Test in Europe Conf. (DATE)*, 2025.

[9] B. Wile, J. C. Goss, and W. Roesner, *Comprehensive Functional Verification: The Complete Industry Cycle*. Morgan Kaufmann, 2005.

[10] E. M. Kim, A. Garg, K. Peng, and N. Garg, "Correlated errors in large language models," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 30 038–30 066.

[11] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.

[12] Z. Yan, W. Fang, M. Li, M. Li, S. Liu, Z. Xie, and H. Zhang, "AssertLLM: Generating hardware verification assertions from design specifications via multi-LLMs," in *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2025.

[13] W. Snyder *et al.*, "Verilator user's guide: Input languages (timing)," https://verilator.org/guide/latest/languages.html, 2024.

[14] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[15] A. C. Cameron and D. L. Miller, "A practitioner's guide to cluster-robust inference," *Journal of Human Resources*, vol. 50, no. 2, pp. 317–372, 2015.

[16] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[17] K.-Y. Liang and S. L. Zeger, "Longitudinal data analysis using generalized linear models," *Biometrika*, vol. 73, no. 1, pp. 13–22, 1986.

[18] K. Pearson, "On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling," *Philosophical Magazine*, vol. 50, no. 302, pp. 157–175, 1900.